In [1]:
```
#Creating TREE
class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
        self.left=None

root=Node(5)
root.right=Node(4)
root.left=Node(3)
root.left.right=Node(2)

print(root.left.right.data)
```

```
2
```

In [ ]:
```
#PREORDER TRAVERSAL, root-->left-->rit\ght
class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
        self.left=None

def preorder(root):
    if root:
        print(root.data,end=" ")
        preorder(root.left)
        preorder(root.right)

root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)

preorder(root)
```

```
10 5 2 7 15
```

In [ ]:
```
#POSTORDER TRAVERSAL, left-->right-->root
class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
        self.left=None

def postOrder(root):
    if root:

        postOrder(root.left)
        postOrder(root.right)
        print(root.data,end=" ")

root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)
```

```
postOrder(root)
```

```
2 7 5 15 10
```

In [7]:
```
#INORDER TRAVERSAL
class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
        self.left=None

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data,end=" ")
        inorder(root.right)
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)

inorder(root)
```

```
2 5 7 10 15
```

In [1]:
```
#Finding maximum height (Depth) of a tree

class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
        self.left=None

class solution:
    def MaxDepth(self,root):
        #Base case
        if root==None:
            return 0
    #Recursive case
        leftheight=self.MaxDepth(root.left)
        rightheight=self.MaxDepth(root.right)
        return max(leftheight,rightheight)+1

obj=solution()
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)
r=obj.MaxDepth(root)
print(f'Maximum height/depth is:{r}')
```

```
Maximum height/depth is:3
```

In [ ]:
```
#Check wether Tree 1 and Tree 2 are same
class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
```

```
        self.left=None
class solution:
    def is_same(self,root1,root2):
        if root1 is None and root2 is None:
            return True
        if root1 is None or root2 is None:
            return False
        if root1.data != root2.data:
            return False
        return self.is_same(root1.left,root2.left) and self.is_same(root1.right,

obj=solution()
root1=Node(1)
root1.left=Node(2)
root1.right=Node(3)

root2=Node(1)
root2.left=Node(4)
root2.right=Node(3)

r=obj.is_same(root1,root2)
print(f'Tree 1 and Tree 2 are same:{r}')
```

```
Tree 1 and Tree 2 are same:False
```

In [2]:

```
#Search in BINARY SEARCH TREE
#TIme complexity of BST->O(Height)
class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
        self.left=None

class solution:
    def search(self,root,target):
        if root==None:
            return None

        curr=root
        while curr!=None:
            if curr.data==target:
                return curr
            elif target<curr.data:
                curr=curr.left
            else:
                curr=curr.right


obj=solution()
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)

result=obj.search(root,7)
print(f'Result is:{result.data}')
```

```
Result is:7
```

In [32]:
```
#INSERT INTO BST
class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
        self.left=None

class solution:
    def insert(self,root,target):
        NewNode=Node(target)

        if root==None:
            return NewNode

        curr=root
        while curr!=None:
            if target<curr.data:
                if curr.left!=None:
                    curr=curr.left
                else:
                    curr.left=NewNode
                    break
            else:
                if curr.right!=None:
                    curr=curr.right
                else:
                    curr.right=NewNode
                    break
        return root


    def inorder(self,root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

obj=solution()
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)

target=9

res=obj.insert(root,target)
obj.inorder(res)
```

```
2 5 7 9 10 15
```

In [ ]:
```
#BST deletion
class Node:
    def __init__(self,data):
        self.data=data
        self.right=None
        self.left=None
class solution:
    def delete(self,root,target):
        #Base case
```

```
        if root is None:
            return None

        if target<root.data:
            root.left=self.delete(root.left,target)
        elif target>root.data:
            root.right=self.delete(root.right,target)
        else:
            #Case 1:For deleting leaf node
            if root.left is None and root.right is None:
                return None
            #Case 2: One child (right)
            elif root.left is None and root.right is not None:
                return root.right
            #Case 3: One child(left)
            elif root.right is None and root.left is not None:
                return root.left

            #Case 4: Two child present--> We have to find inorder successor.
            # The Inorder Successor is the smallest value in the right subtree o
            else:
                temp=root.right
                while temp.left!=None:
                    temp=temp.left

                root.data=temp.data
                root.right=self.delete(root.right,temp.data)
        return root

    def inorder(self,root):
        if root:
            self.inorder(root.left)
            print(root.data,end=" ")
            self.inorder(root.right)



obj=solution()
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)

print("\nBefore Deletion")
obj.inorder(root)

res=obj.delete(root,5)
print(f"\nAfter Deletion")
obj.inorder(res)
```

```
Before Deletion
2 5 7 10 15
After Deletion
2 7 10 15
```

In [ ]:
```
#VAlidate BTS
class Node:
    def __init__(self,data):
```

```
        self.data=data
        self.right=None
        self.left=None

class solution:
    def check(self,root,mn,mx):
        if root is None:
            return True

        if root.data<mn or root.data>mx:
            return False

        checkLeft=self.check(root.left,mn,root.data-1)
        checkRight=self.check(root.right,root.data+1,mx)

        return checkLeft and checkRight


    def validateBST(self,root):
        return self.check(root,-100000000000,100000000000)

obj=solution()
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)

r=obj.validateBST(root)
print(f'The given Tree is BST: {r}')
```

```
The given Tree is BST: True
```

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click here for more info.

View Jupyter log for further details.